How the Embedding Model Learns

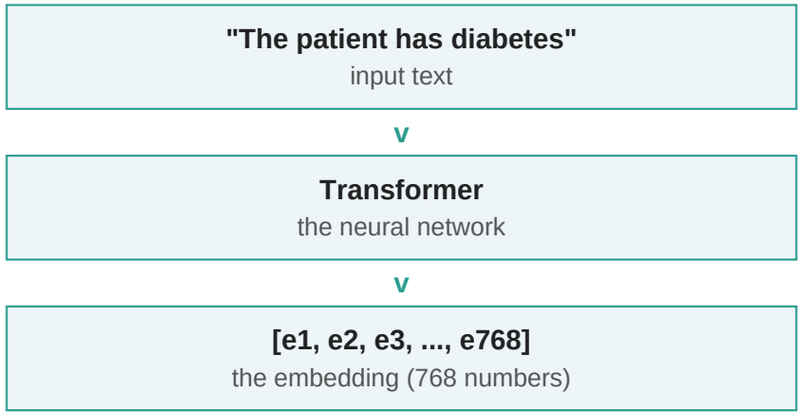
Math, training and the 768-dimensional embedding

> **Model used in these notes:** `NeuML/bioclinical-modernbert-base-embeddings`. The training details below come from its model card.

1. The basic math idea

The model learns one function. It takes text and gives back a list of 768 numbers:

```
f(text) = e          where e has 768 numbers  (e is in R^768)
```



These 768 values are the **coordinates of the sentence** in a 768-dimensional space. To compare two sentences, we use **cosine similarity**:

```
                    A . B
cosine(A, B)  =  --------------
                 ||A|| x ||B||
```

> **Simple meaning:** If two sentences point in a similar direction in this 768-dimensional space, they are semantically similar. This model was trained with cosine similarity as its similarity function.

2. Loss function: MultipleNegativesRankingLoss

A **loss function** is a score that tells the model how wrong it is. Training tries to make this score smaller. This model uses **MultipleNegativesRankingLoss** with these settings:

Setting	Value
Similarity	Cosine similarity
Scale	20

Imagine one training batch of question and answer pairs:

Question	Correct answer
diabetes treatment	Treatment options for diabetes
heart disease	Cardiovascular disease...
cancer symptoms	Common symptoms of cancer...

- Each question and its correct answer should have **HIGH** similarity.
- The other answers in the batch act as **negative examples**. For example, "diabetes treatment" and the heart disease answer should have **LOW** similarity.

> **Heart of the model:** Pull related texts closer together and push unrelated texts apart.

3. Activation function: GELU

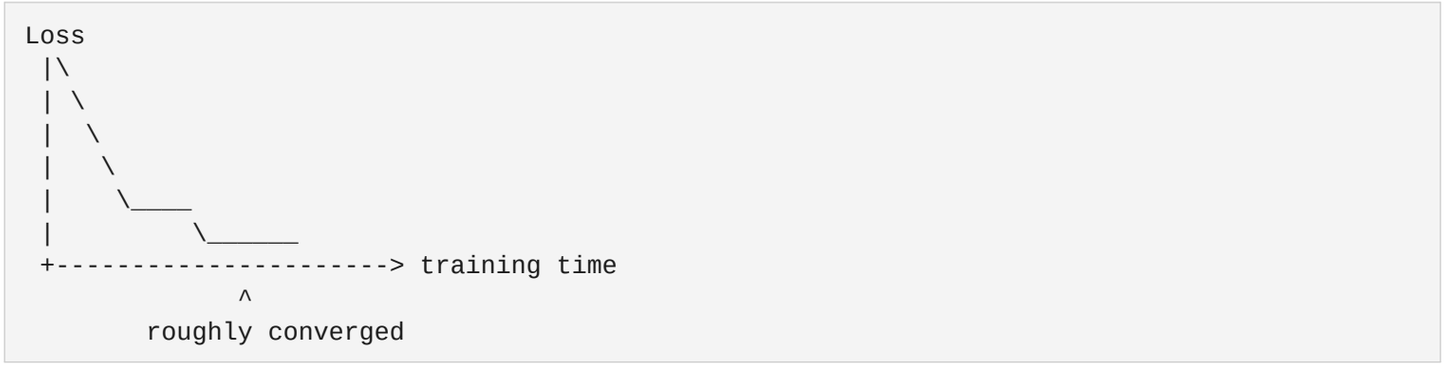
The ModernBERT configuration uses **GELU** as its hidden activation. It is applied to the numbers inside the Transformer:

```
numbers  ->  GELU  ->  transformed numbers
```

GELU has a similar purpose to ReLU. It adds **non-linearity**, which lets the network learn complicated relationships. You do not need to memorize the equation yet.

4. Convergence

Convergence is **not** a special activation or equation. It describes what happens during training: the model's parameters slowly reach a point where the loss stops improving much.



The published training settings for this model:

Setting	Value	What it means
Epochs	1	One full pass over the training data
Optimizer	AdamW	The method that updates the parameters
Learning rate	2e-5	How big each update step is
Warmup	10,000 steps	Start with tiny steps, then increase slowly
Scheduler	WarmupLinear	Learning rate rises during warmup, then falls in a straight line
Weight decay	0.01	A small penalty that keeps parameters from growing too large

> **Careful:** One epoch was simply the **chosen** training setup. It does not prove the model converged perfectly. Convergence is about how the loss behaves during training.

5. The whole story in two flows

Architecture of the model:

Part	Value
Base model	ModernBERT
Layers	22
Attention heads	12
Hidden size	768
Pooling	Mean pooling

Using the model (inference)

Text
v
Tokenizer
v
Token IDs
v
ModernBERT
attention + layers, GELU
v
Token representations
v
Mean pooling
v
768-D vector
v
Cosine similarity

Training the model

Related text pair
v
Two 768-D vectors
v
Cosine similarity
v
MultipleNegatives-RankingLoss
v
AdamW
adjusts millions of parameters
v
Repeat
next batch, go back to start

6. Quick summary

- The model learns a function from text to **768 numbers**.
- Similarity is measured with **cosine similarity**.
- Loss: **MultipleNegativesRankingLoss** (scale 20) pulls matching pairs together and pushes others apart.
- Activation: **GELU**, which adds non-linearity.
- Training: **AdamW**, learning rate 2e-5, 1 epoch, 10,000 warmup steps.
- **Convergence** means the loss stops improving much. One epoch does not guarantee it.

> **One sentence to remember:** This model learns a 768-dimensional space where related biomedical texts are encouraged to be close together, using cosine similarity and MultipleNegativesRankingLoss during training.